3-dimensional arrays:

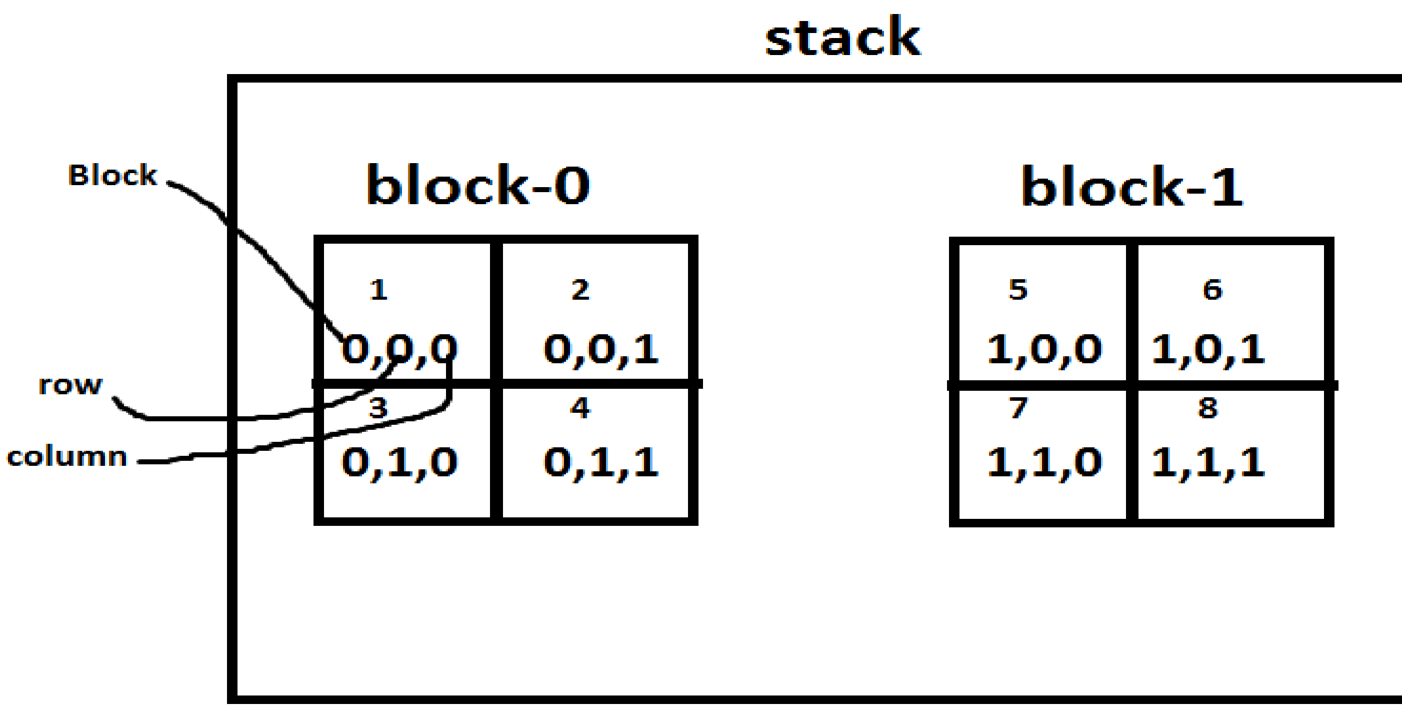
An array with several blocks, rows and columns.

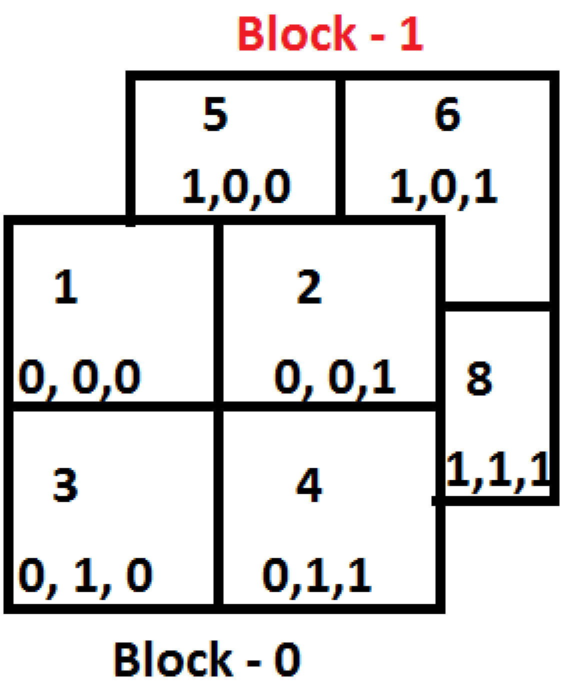
An array with 3 subscripting operators [ ] [ ] [ ].

Syntax:

datatype variable [blocks] [rows] [columns];

Eg: int a[2][2][2]={1,2,3,4,5,6,7,8};





Eg:

rows

int a[2][2][2]; an array with 3 subscripts [ ].

blocks columns

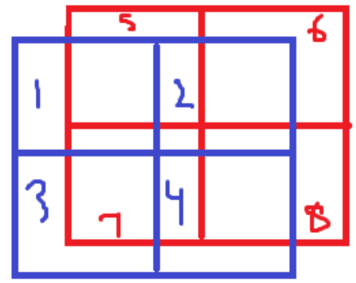
a[0][0][0]

b -0

1	2
3	4

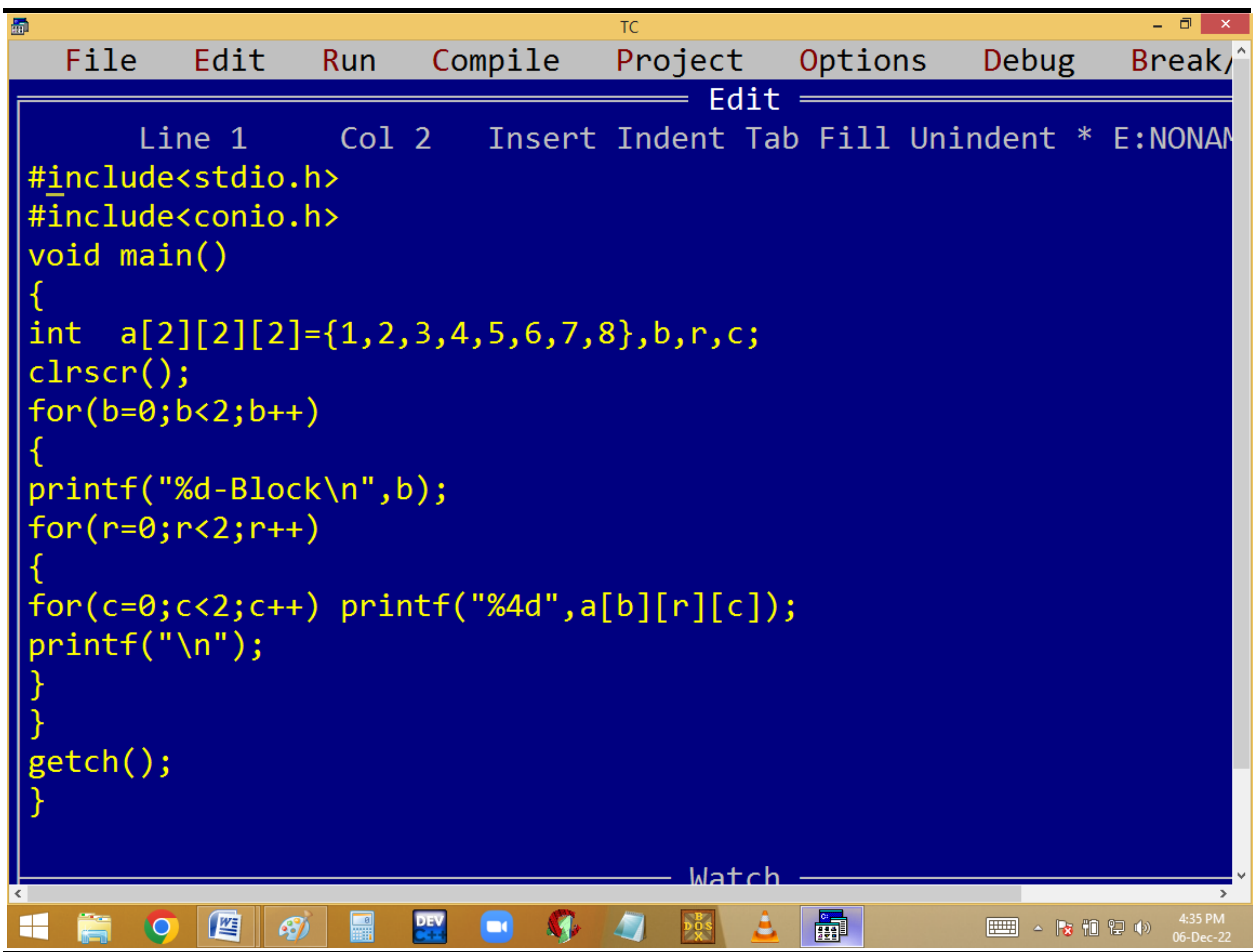
b-1

5	6
7	8

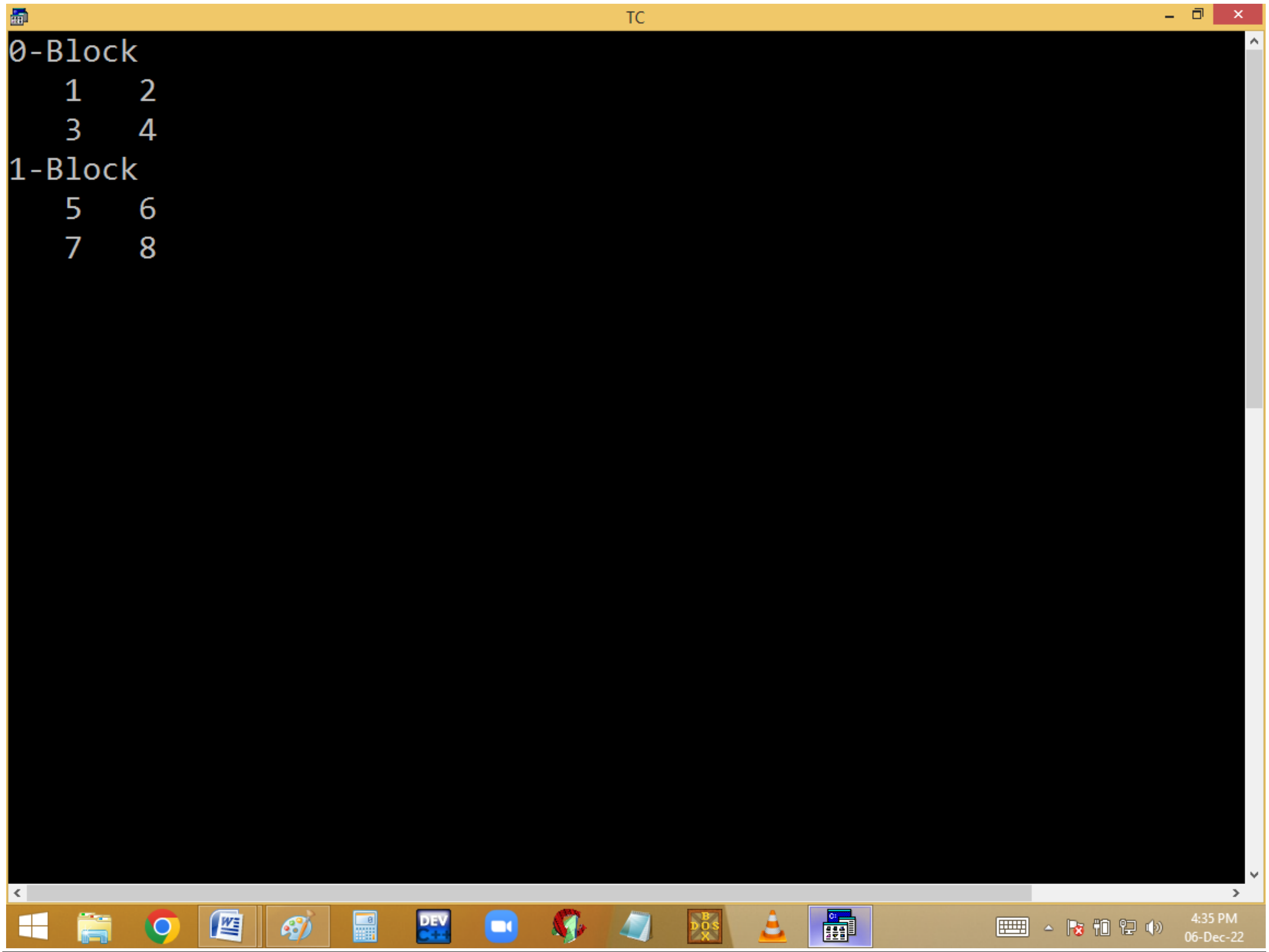


eg: int class[2][60][6];

datatype class[sections][stus][marks];




```
#include<stdio.h>
#include<conio.h>
void main()
{
int  a[2][2][2]={1,2,3,4,5,6,7,8},b,r,c;
clrscr();
for(b=0;b<2;b++)
{
printf("%d-Block\n",b);
for(r=0;r<2;r++)
{
for(c=0;c<2;c++) printf("%4d",a[b][r][c]);
printf("\n");
}
}
getch();
}
```



4-dimensional array:

An array with several sets, blocks, rows and columns.

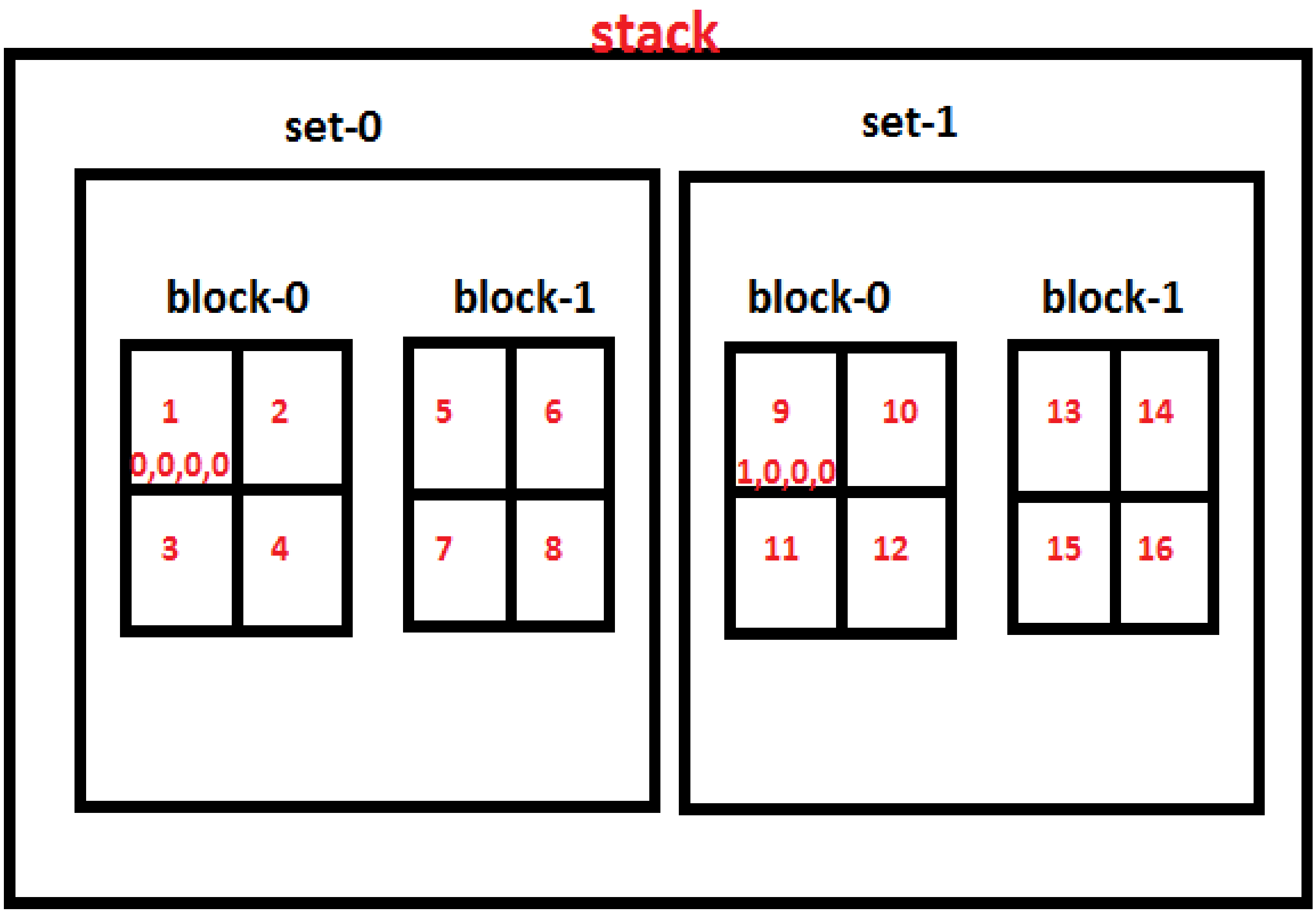
An array with 4 subscripting operators [ ] [ ] [ ] [ ] .

Syntax:

datatype variable [sets] [blocks] [rows] [cols];

eg:

int a[2] [2] [2] [2]= {1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16};



datatype var[class][sec][stu][marks];

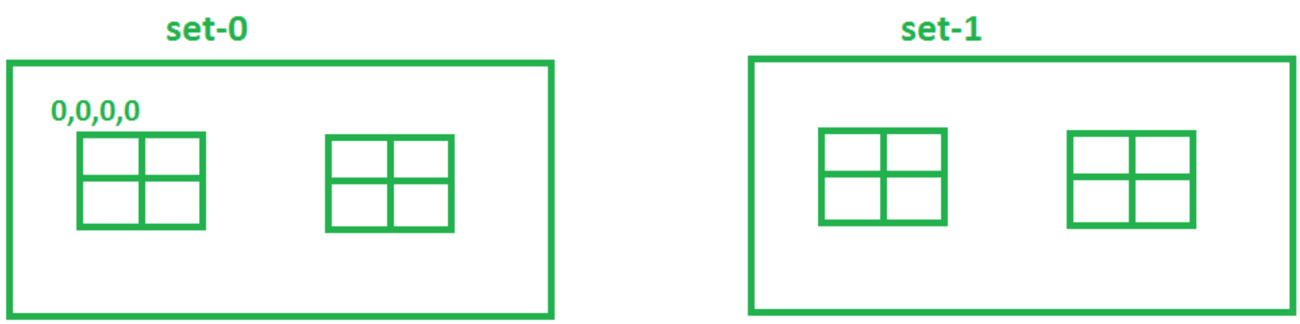
int school[5][2][60][6];

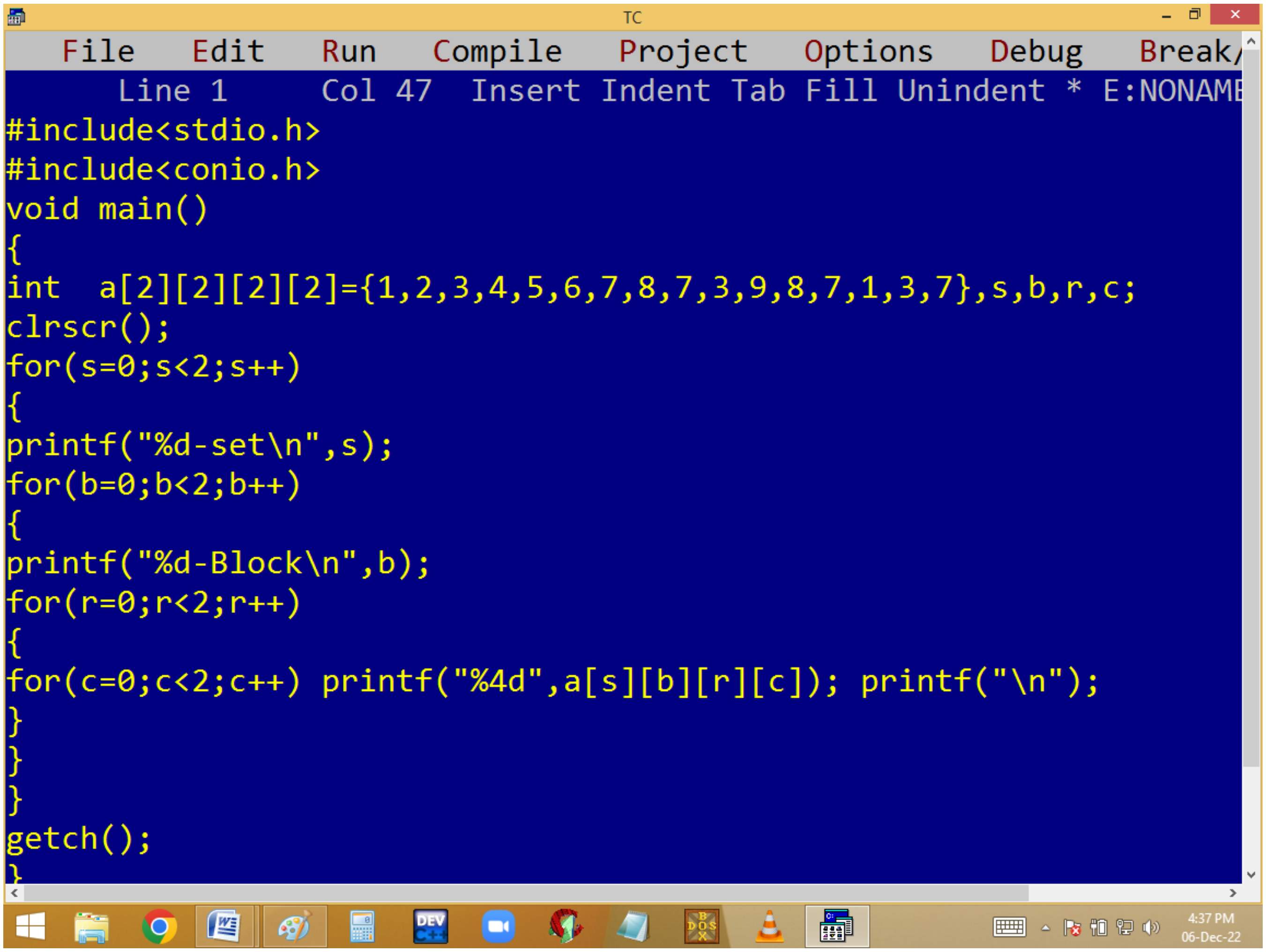
rows

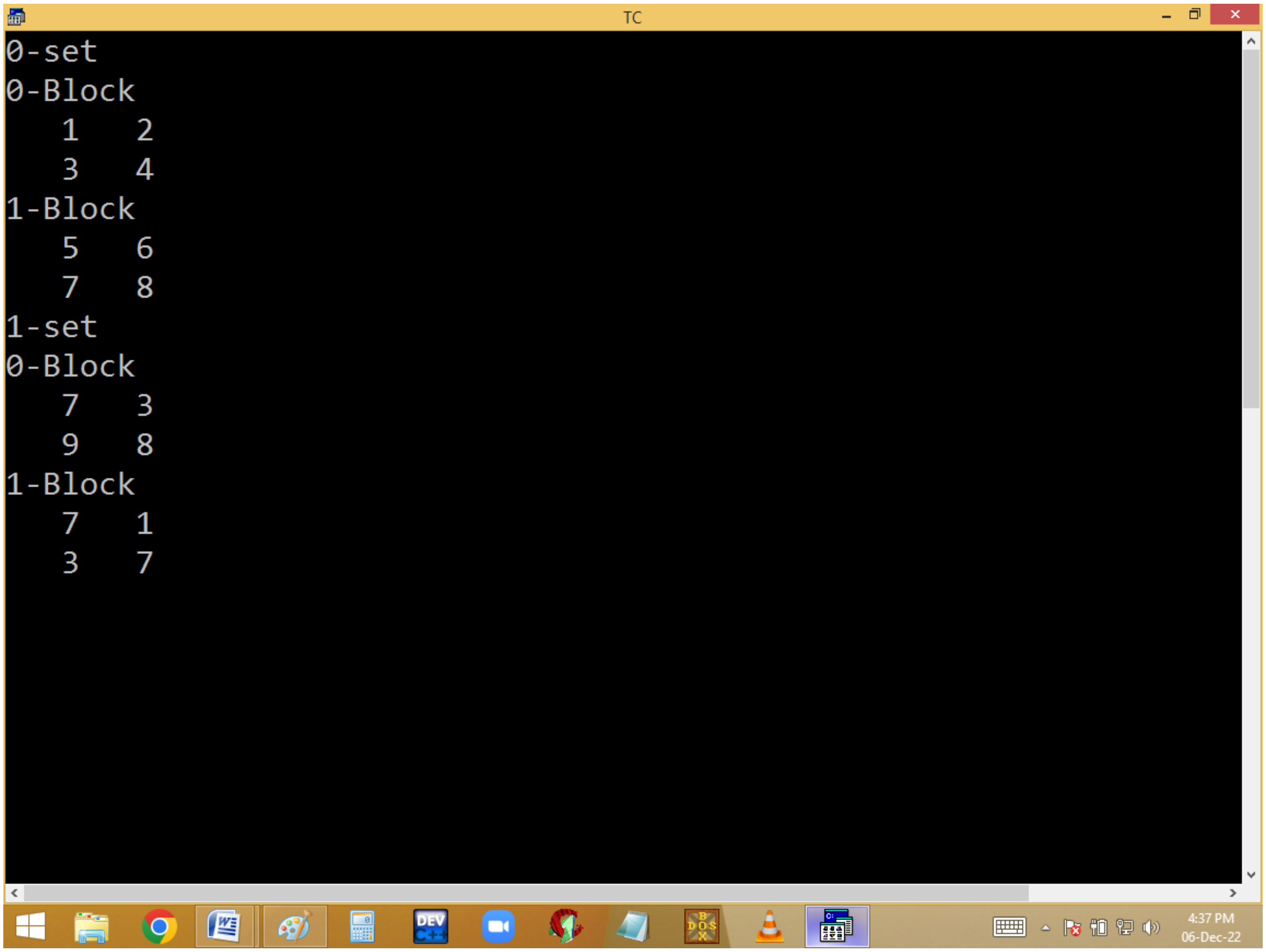
set

int a[2][2][2][2]; an array with 3 subscripts [ ].

blocks columns




TC
File Edit Run Compile Project Options Debug Break/
Line 1 Col 47 Insert Indent Tab Fill Unindent * E:NONAME
#include<stdio.h>
#include<conio.h>
void main()
{
int a[2][2][2][2]={1,2,3,4,5,6,7,8,7,3,9,8,7,1,3,7},s,b,r,c;
clrscr();
for(s=0;s<2;s++)
{
printf("%d-set\n",s);
for(b=0;b<2;b++)
{
printf("%d-Block\n",b);
for(r=0;r<2;r++)
{
for(c=0;c<2;c++) printf("%4d",a[s][b][r][c]); printf("\n");
}
}
}
getch();
}
4:37 PM
06-Dec-22

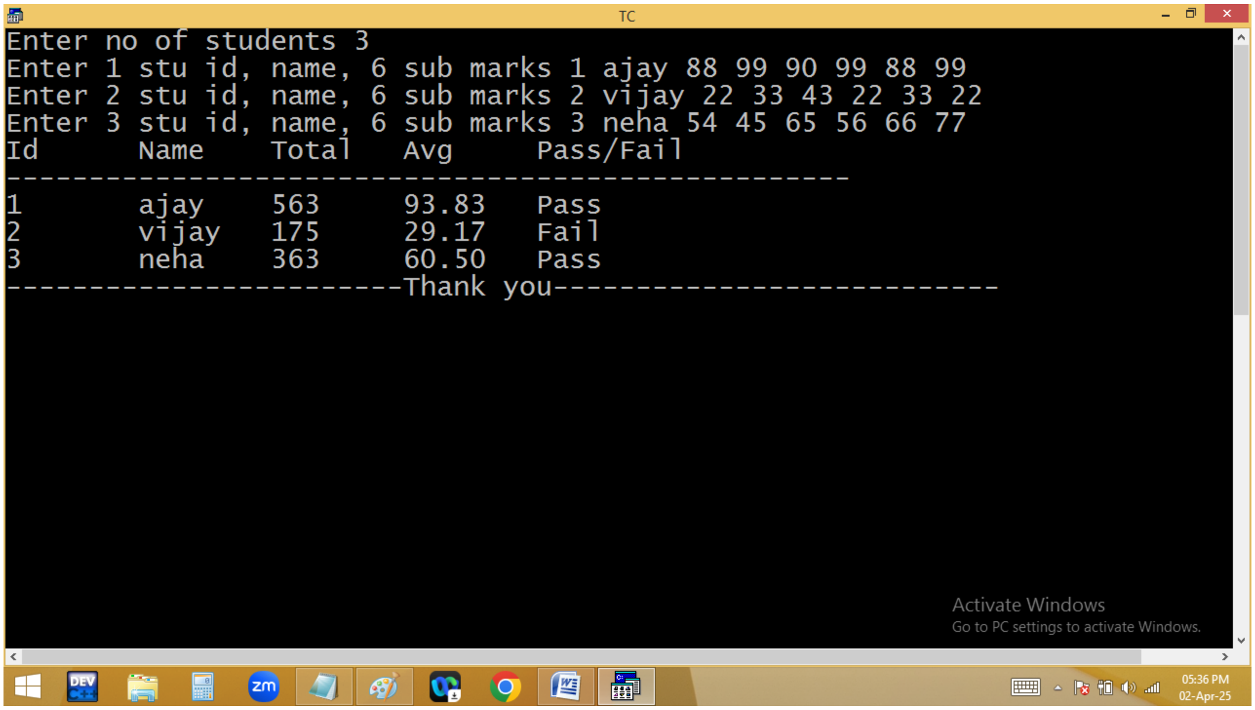


Finding n students grade

```
#include<stdio.h>

#include<conio.h>

void tummy(float a){float *p=&a;}

void main()
```

```
{
float  stu[10][10]={0};
char   name[10][30];
int  n,i,p,j;
clrscr();
printf("Enter no of students ");scanf("%d",&n);
for(i=0;i<n;i++)
{
printf("Enter %d stu id, name, 6 sub marks ", i+1);
scanf("%f%s",&stu[i][0],name[i]);
for(j=1;j<=6;j++)
{scanf("%f",&stu[i][j]);
stu[i][7]+=stu[i][j];
if(stu[i][j]<35)stu[i][9]=-1;
}
stu[i][8]=stu[i][7]/6.0;
}
```

```
puts("Id\tName\tTotal\tAvg\tPass/Fail");
puts("---------------------------------------------------");
for(i=0;i<n;i++)
{
printf("%.0f\t%s\t%.0f\t%.2f\t%s\n",stu[i][0],name[i],stu[i][7],stu[i][8],
stu[i][9]==0?"Pass":"Fail");
}
puts("------------------------Thank you---------------------------");
getch();
}
```



STRINGS

- A group of characters is called string.
- It is one dimensional character array.
- It is alpha-numeric.
- It is an implicit pointer.
- It is a derived data type.

Note:

- **One byte should be left for Null char(\0). Otherwise we are getting garbage or junk values. Null char indicates string is completed.**
- **String variable Size can't be less than string. Otherwise we are getting error.**
- **Using = operator, we can't copy a string into another. We have to use strcpy() or copy character by character manually.**
- **Using == (comparison) operator, we can't compare two strings. Use strcmp() or compare the characters one by one manually.**

Syntax:

char variable [size] = "string";

or

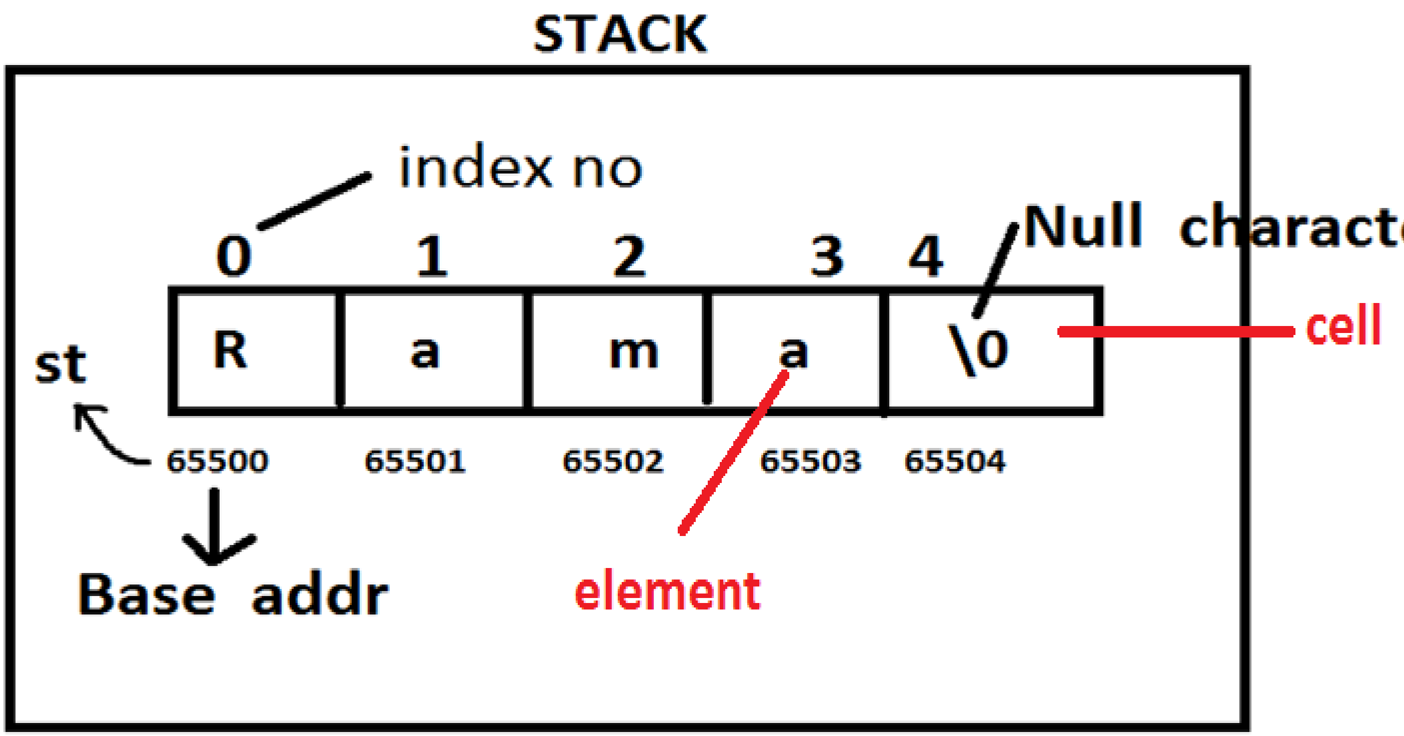
char variable[ ]="string";

Eg: char st[5] = "Rama";

Why?
In C, a string is an array of characters stored in memory.

When you use == to compare two strings, it compares their memory addresses, not their actual contents.

Since the two strings are stored at different memory locations, str1 == str2 will return false, even if the contents are the same.



Note: String is implicit pointer because of string variable stores base address.

String declaration methods:

char st [5] =”rama”; Ok

char st [20] =” Naresh It”; Ok

char st [4] = { ‘r’, ‘a’, ‘m’ }; Ok → char array.

char st[3]= “ram”; It gives garbage values in printing.

char   st [3] = “rama”; error

char st[0]; error

char st[0]=”abc”; Ok

char st[-5]; error

char st[5.5]; error

char   st[5%3];   Ok   → char st[2];

char st[3+2]; → st[5] → Ok

char st[ ] =”Ram”; Ok.

char st[ ] ; error

int n=20;

char st[n]; No

#define   n 20

char st[n]; Ok

Note: String variable size always constant positive integer value.

in turbo if the size is even then it is properly show the output, but if you enter odd number then output with garbage value print